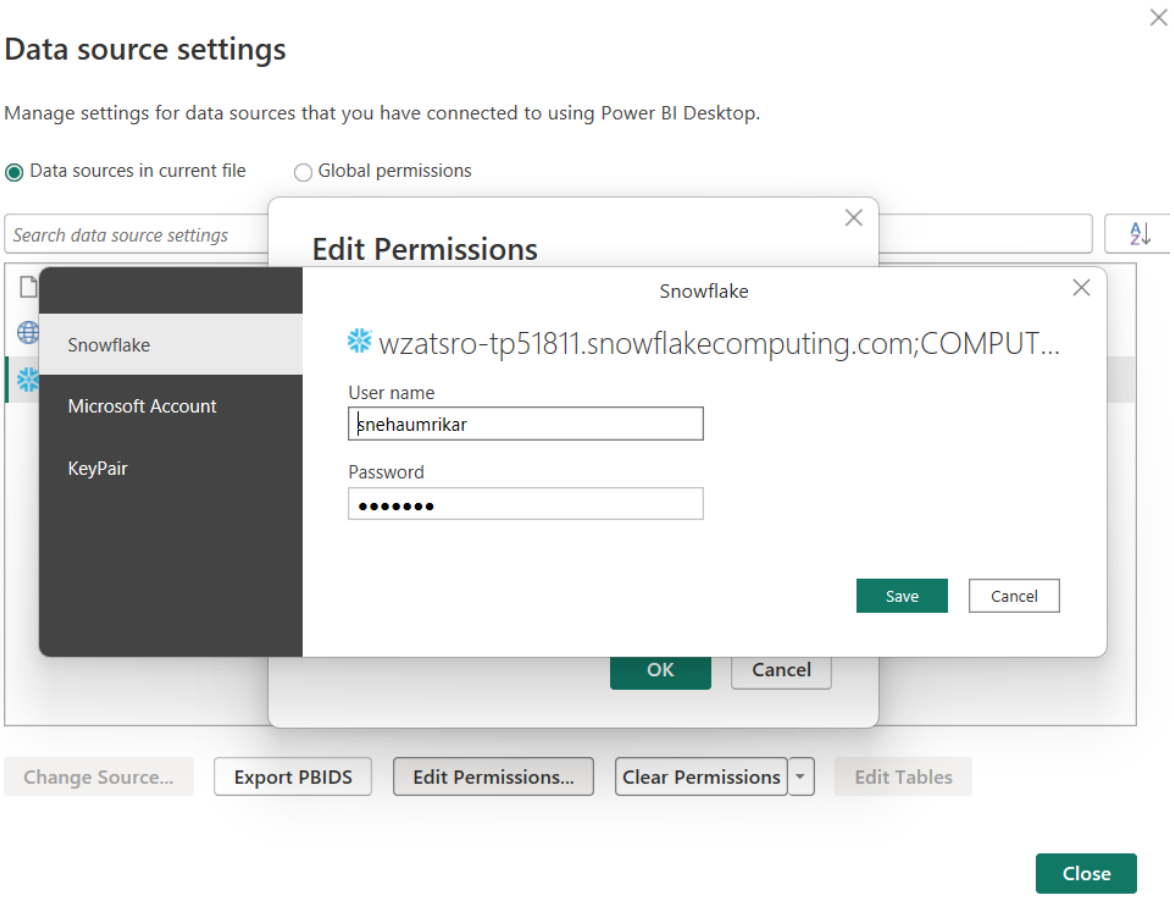
Steps ETL

1. Download dataset from Kaggle use below link
   https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
2. Cop paste the csv files in the project folder. In Pycharm or Google colab
   Install snowflake connector package using below command
   pip install snowflake-connector-python
3. Connection to Snowflake, creating warehouse, schema, tables, staging area
   is done through **snowflake_setup.py** file
4. Loading data from csv files is done in **extract_data.py** file
5. Cleaning and transformation of data is done in **clean_and_transform_data.py**
6. Loading data to Snowflake is done in **load_data_to_snowflake.py** file
7. All the ETL functions are called one after other in **call_etl_function.py** file
   Output of Step 7 and Step 8 is shown in the end
8. Run Post Validation Checks are carried to ensure all the data is entered in snowflake correctly. It checks row counts, if tables have any null value, non positive value for price and freight_value, invalid_review, referential integrity checks to identify orphan record basically comparison is done between fact table and dimension table

After data is loaded successfully in Snowflake in following steps are performed in Power BI

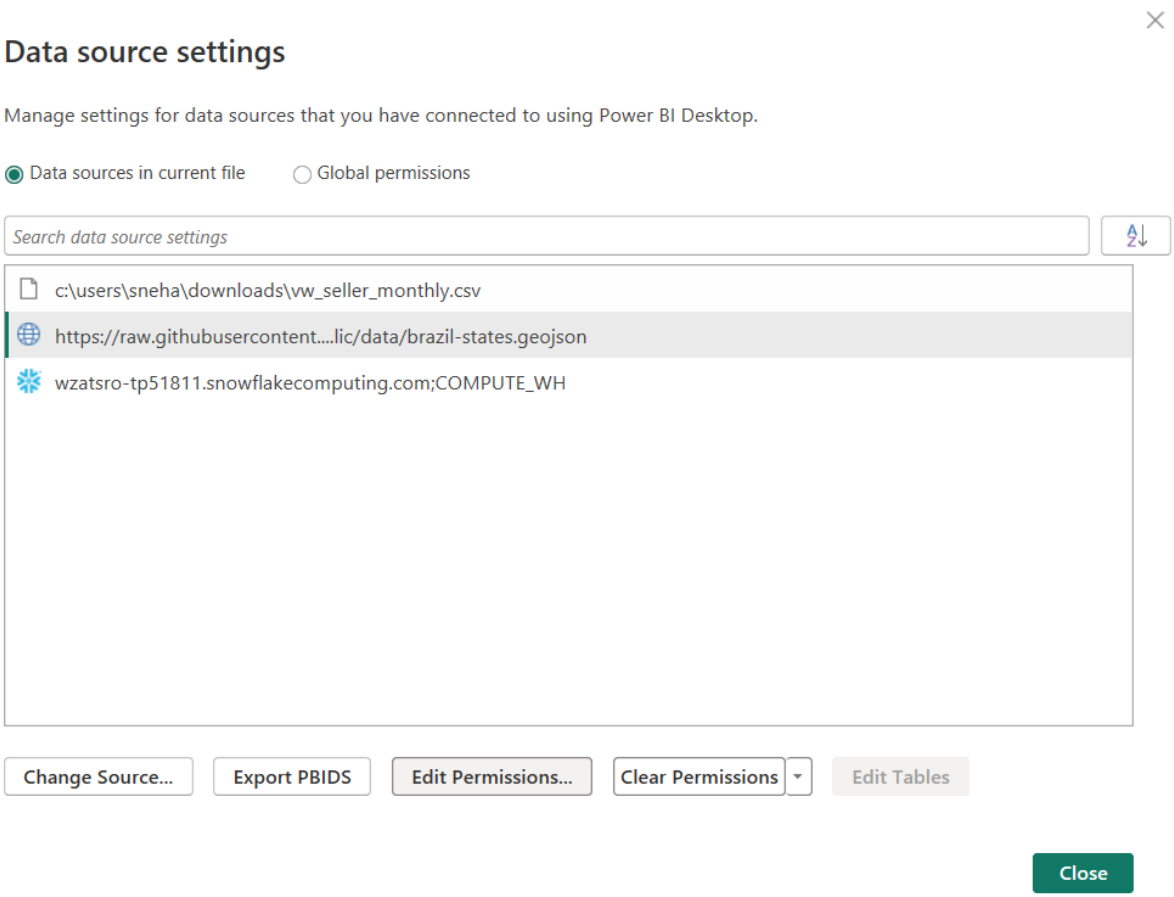
Steps Power BI

1. Got Data from Snowflake into Power BI directly by making connections using Get Data -> Snowflake -> Entering user name password, Dataware house name

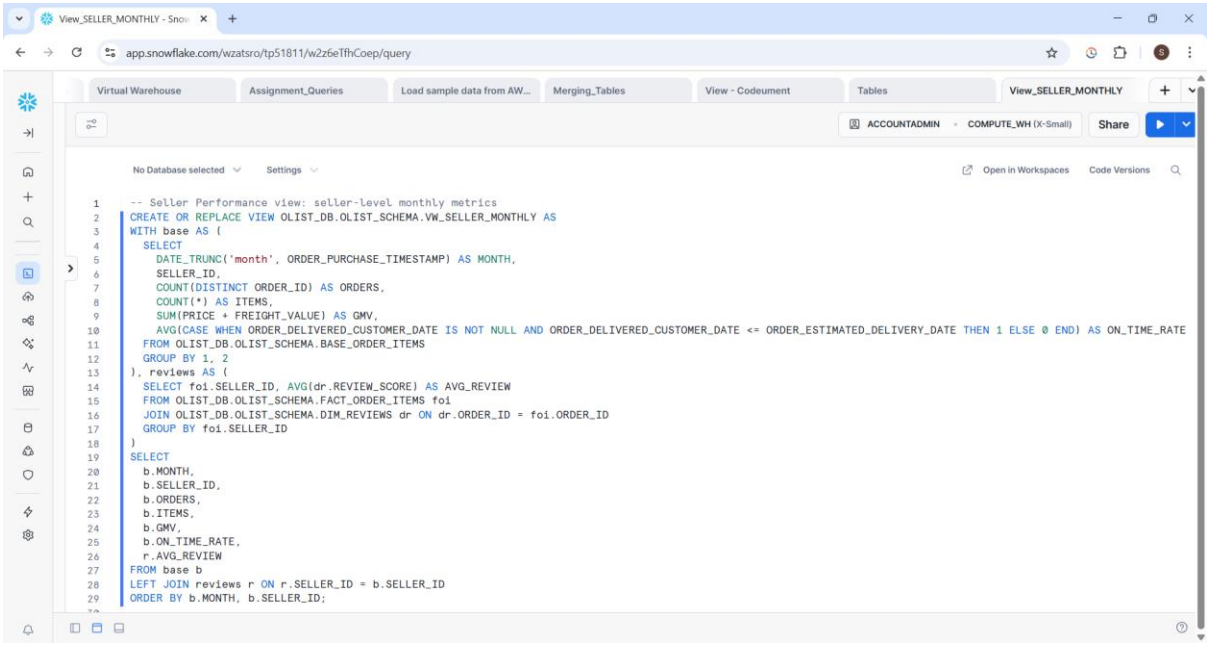


2. Also loaded JSON file from below link into power bi, converted it to brazil_states table, to map the abbreviated State name to actual state map for using in map visuals, for example to map ‘SP’ to ‘Sao Paulo’

https://raw.githubusercontent.com/codeforamerica/click_that_hood/master/public/data/brazil-states.geojson



3. Created and executed SQL query in Snowflake to create a View to filter out the monthly seller data like average reviews, orders, on_time_rate. Added data in the dashboard



4. Created following DAX measures

```
Customer count = COUNT(DIM_CUSTOMERS[CUSTOMER_ID])
First Purchase Date =
CALCULATE(
   MIN('DIM_ORDERS'[ORDER_PURCHASE_TIMESTAMP]),
   ALLEXCEPT(DIM_ORDERS, 'DIM_ORDERS'[CUSTOMER_ID])
)
Active Customers = DISTINCTCOUNT('DIM_ORDERS'[CUSTOMER_ID])
Active Sellers = DISTINCTCOUNT('FACT_ORDER_ITEMS'[SELLER_ID])
Average Order Value = [Gross Merchandise Value] / [Total Orders]
Gross Merchandise Value = SUMX(FACT_ORDER_ITEMS,
'FACT_ORDER_ITEMS'[PRICE] + 'FACT_ORDER_ITEMS'[FREIGHT_VALUE])
On Time Delivery % = DIVIDE(CALCULATE(COUNTROWS(DIM_ORDERS),
'DIM_ORDERS'[ORDER_ESTIMATED_DELIVERY_DATE] <=
'DIM_ORDERS'[ORDER_ESTIMATED_DELIVERY_DATE]),
CALCULATE(COUNTROWS(DIM_ORDERS),
NOT(ISBLANK('DIM_ORDERS'[ORDER_DELIVERED_CUSTOMER_DATE]))))
Total Orders = DISTINCTCOUNT('DIM_ORDERS'[ORDER_ID])
```

5. Transform data

DIM_CUSTOMER

DIM_CUSTOMER merged with brazil_states to add state names in the Customer table for all the state name abbreivations, added only the state names into the DIM_CUSTOMERS

Note: Removed the accent from City names were removed from this table during data cleaning process

DIM_ORDERS

Added Delivery_Delay_Days custom column, renamed, changed type to int

```
if [ORDER_DELIVERED_CUSTOMER_DATE] = null or
[ORDER_ESTIMATED_DELIVERY_DATE] = null
then null
else Duration.Days([ORDER_DELIVERED_CUSTOMER_DATE] -
[ORDER_ESTIMATED_DELIVERY_DATE]))
```

Added Bucket_Delay custom column

```
if [Delivery_Delay_Days] = null then "Not Delivered"'
else if [Delivery_Delay_Days] < -3 then "Very Early"'
else if [Delivery_Delay_Days] < 0 then "Early"'
else if [Delivery_Delay_Days] = 0 then "On Time"'
else if [Delivery_Delay_Days] <= 3 then "Slightly Late"'
else if [Delivery_Delay_Days] <= 7 then "Late"' else "Very Late"')
```

DIM_PRODUCTS

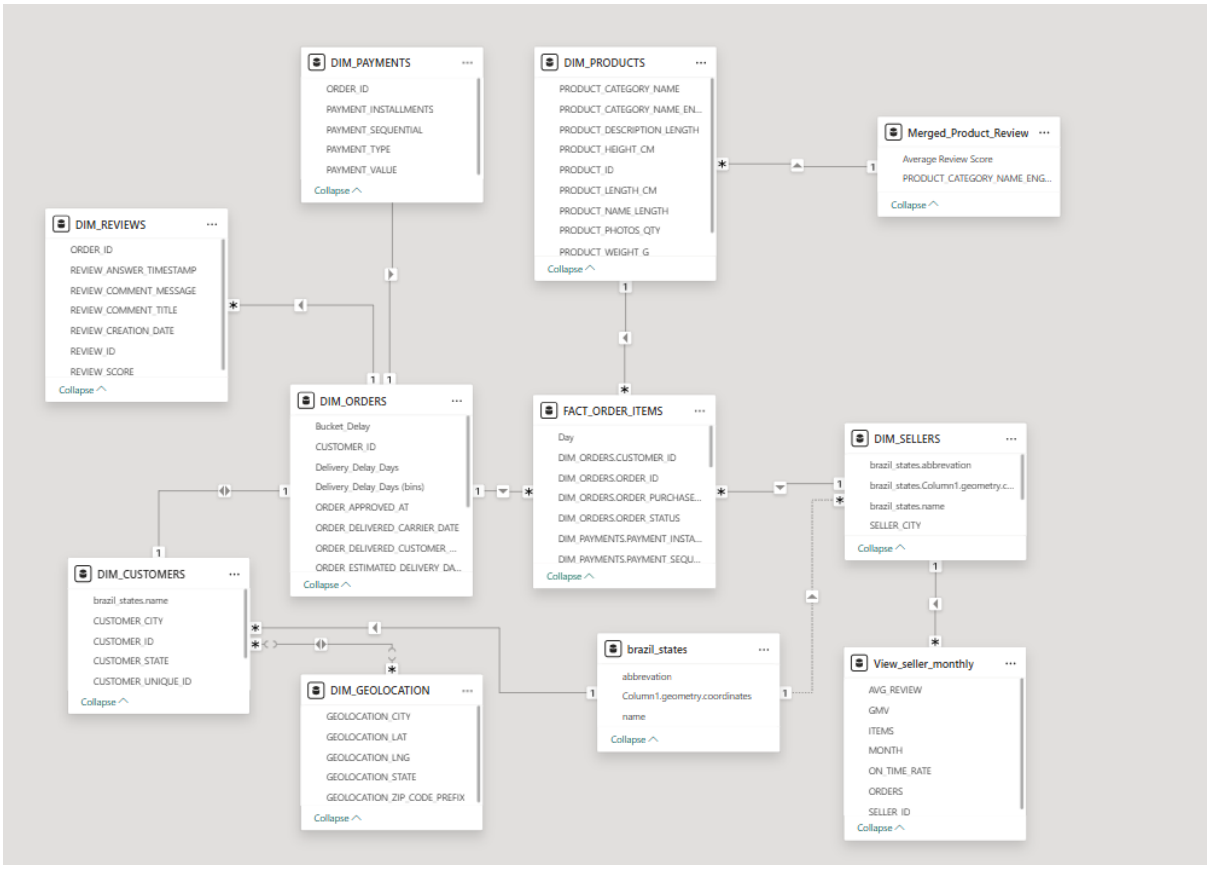
To name the product names readable replaced underscores, capitalized first letter
Note: Before adding the products data in to warehouse, I have merged the Products and Product English name dataframe to get the english names for the products. Added missing English names for two products

DIM_SELLER
Merged it with brazil_state to map the abbreviated State name to actual state map for each seller fo using in map visuals

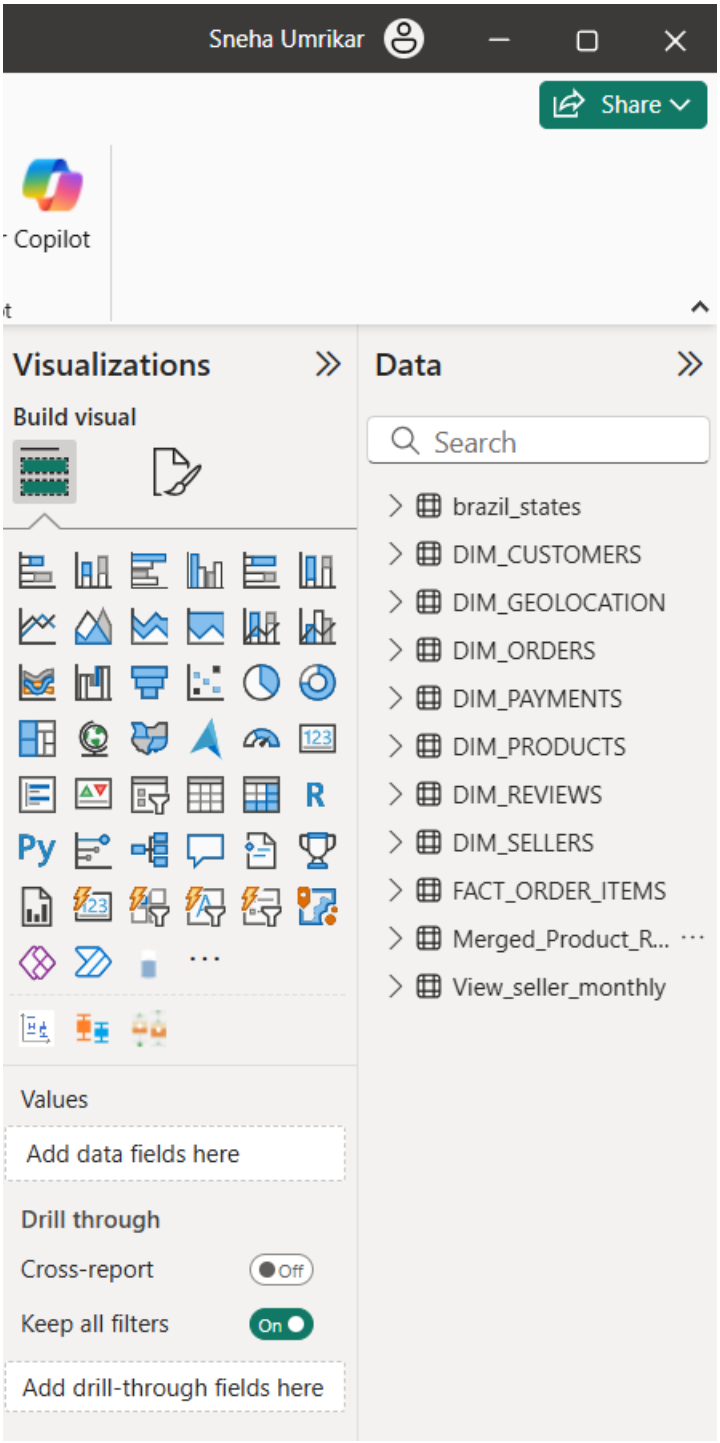
FACT_ORDER_ITEMS
Merged it with DIM_Orders to get order purchase date and time,
Added custom column to extract month from that date
Table.TransformColumns(#"Added Custom",{{"Month", Date.Month, Int64.Type}})
Added custom column to extract day from that date
= Table.TransformColumns(#"Added Custom1", {{"Day", each Date.DayOfWeekName(_), type text}})
Merged it with DIM_PAYMENTS to get payment detail for each ordered item

Merged DIM_PRODUCTS AND DIM_REVIEWS into another Table "Merged_Product_Review" and then grouped by Product_category to get average review on the product category.

6. The final data model looks like below



7. Data is ready for visualization



Output of Step 7

Attempting to connect to Snowflake to set up database...

Successfully connected to Snowflake!

Creating warehouse: MBI_DW...

Warehouse MBI_DW created or already exists.

Using warehouse: MBI_DW...

Warehouse MBI_DW is now in use.

Creating database: OLIST_DB...

Database OLIST_DB created or already exists.

Creating schema: OLIST_SCHEMA...

Schema OLIST_SCHEMA created or already exists.

Creating tables...

Table Dim_Customers created or already exists.

Table Dim_Products created or already exists.

Table Dim_Sellers created or already exists.

Table Dim_Orders created or already exists.

Table Dim_Payments created or already exists.

Table Dim_Reviews created or already exists.

Table Dim_Geolocation created or already exists.

Table Fact_Order_Items created or already exists.

Creating internal stage: csv_upload_stage...

Internal stage csv_upload_stage created or already exists.

Snowflake setup committed successfully.

Snowflake cursor closed.

Snowflake connection closed after setup.

	customer_id	customer_unique_id \
0	06b8999e2fba1a1fbc88172c00ba8bc7	861eff4711a542e4b93843c6dd7febb0
1	18955e83d337fd6b2def6b18a428ac77	290c77bc529b7ac935b93aa66c333dc3
2	4e7b3e00288586ebd08712fdd0374a03	060e732b5b29e8181a18229c7b0b2b5e
3	b2b6027bc5c5109e529d4dc6358b12c3	259dac757896d24d7702b9acbbff3f3c

4 4f2d8ab171c80ec8364f7c12e35b23ad 345ecd01c38d18a9036ed96c73b8d066

```
   customer_zip_code_prefix          customer_city customer_state
0                     14409                 franca             SP
1                      9790  sao bernardo do campo              SP
2                      1151              sao paulo             SP
3                      8775        mogi das cruzes              SP
4                     13056               campinas             SP
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99441 entries, 0 to 99440
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   customer_id             99441 non-null  object
 1   customer_unique_id      99441 non-null  object
 2   customer_zip_code_prefix  99441 non-null  int64
 3   customer_city           99441 non-null  object
 4   customer_state          99441 non-null  object
dtypes: int64(1), object(4)
memory usage: 3.8+ MB
None
       customer_zip_code_prefix
count              99441.000000
mean               35137.474583
std                29797.938996
min                 1003.000000
25%                 11347.000000
```

50% 24416.000000

75% 58900.000000

max 99990.000000

customer_id 0

customer_unique_id 0

customer_zip_code_prefix 0

customer_city 0

customer_state 0

dtype: int64

Empty DataFrame

Columns: [customer_id, customer_unique_id, customer_zip_code_prefix, customer_city, customer_state]

Index: []

	product_id	product_category_name
0	1e9e8ef04dbcff4541ed26657ea517e5	perfumaria
1	3aa071139cb16b67ca9e5dea641aaa2f	artes
2	96bd76ec8810374ed1b65e291975717f	esporte_lazer
3	cef67bcfe19066a932b7673e239eb23d	bebes
4	9dc1a7de274444849c219cff195d0b71	utilidades_domesticas

	product_name_lenght	product_description_lenght	product_photos_qty
0	40.0	287.0	1.0
1	44.0	276.0	1.0
2	46.0	250.0	1.0
3	27.0	261.0	1.0
4	37.0	402.0	4.0

```
   product_weight_g  product_length_cm  product_height_cm  product_width_cm
0             225.0               16.0               10.0              14.0
1            1000.0               30.0               18.0              20.0
2             154.0               18.0                9.0              15.0
3             371.0               26.0                4.0              26.0
4             625.0               20.0               17.0              13.0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32951 entries, 0 to 32950
Data columns (total 9 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   product_id              32951 non-null  object
 1   product_category_name   32341 non-null  object
 2   product_name_lenght     32341 non-null  float64
 3   product_description_lenght  32341 non-null  float64
 4   product_photos_qty      32341 non-null  float64
 5   product_weight_g        32949 non-null  float64
 6   product_length_cm       32949 non-null  float64
 7   product_height_cm       32949 non-null  float64
 8   product_width_cm        32949 non-null  float64
dtypes: float64(7), object(2)
memory usage: 2.3+ MB
None
       product_name_lenght  product_description_lenght  product_photos_qty  \
count         32341.000000                32341.000000        32341.000000
mean             48.476949                  771.495285            2.188986
std              10.245741                  635.115225            1.736766
```

min	5.000000	4.000000	1.000000
25%	42.000000	339.000000	1.000000
50%	51.000000	595.000000	1.000000
75%	57.000000	972.000000	3.000000
max	76.000000	3992.000000	20.000000

	product_weight_g	product_length_cm	product_height_cm	\
count	32949.000000	32949.000000	32949.000000	
mean	2276.472488	30.815078	16.937661	
std	4282.038731	16.914458	13.637554	
min	0.000000	7.000000	2.000000	
25%	300.000000	18.000000	8.000000	
50%	700.000000	25.000000	13.000000	
75%	1900.000000	38.000000	21.000000	
max	40425.000000	105.000000	105.000000	

	product_width_cm
count	32949.000000
mean	23.196728
std	12.079047
min	6.000000
25%	15.000000
50%	20.000000
75%	30.000000
max	118.000000
product_id	0
product_category_name	610

product_name_lenght 610

product_description_lenght 610

product_photos_qty 610

product_weight_g 2

product_length_cm 2

product_height_cm 2

product_width_cm 2

dtype: int64

Empty DataFrame

Columns: [product_id, product_category_name, product_name_lenght, product_description_lenght, product_photos_qty, product_weight_g, product_length_cm, product_height_cm, product_width_cm]

Index: []

<class 'pandas.core.frame.DataFrame'>

Index: 32336 entries, 0 to 32950

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	product_id	32336 non-null	object
1	product_category_name	32336 non-null	object
2	product_name_length	32336 non-null	float64
3	product_description_length	32336 non-null	float64
4	product_photos_qty	32336 non-null	float64
5	product_weight_g	32336 non-null	float64
6	product_length_cm	32336 non-null	float64
7	product_height_cm	32336 non-null	float64
8	product_width_cm	32336 non-null	float64

dtypes: float64(7), object(2)

memory usage: 2.5+ MB

None

product_id 0

product_category_name 0

product_name_length 0

product_description_length 0

product_photos_qty 0

product_weight_g 0

product_length_cm 0

product_height_cm 0

product_width_cm 0

dtype: int64

product_id 32336

product_category_name 73

product_name_length 66

product_description_length 2960

product_photos_qty 19

product_weight_g 2201

product_length_cm 99

product_height_cm 102

product_width_cm 95

dtype: int64

['perfumaria' 'artes' 'esporte_lazer' 'bebes' 'utilidades_domesticas'
 'instrumentos_musicais' 'cool_stuff' 'moveis_decoracao'
 'eletrodomesticos' 'brinquedos' 'cama_mesa_banho'
 'construcao_ferramentas_seguranca' 'informatica_acessorios'
 'beleza_saude' 'malas_acessorios' 'ferramentas_jardim'

'moveis_escritorio' 'automotivo' 'eletronicos' 'fashion_calcados'
'telefonia' 'papelaria' 'fashion_bolsas_e_acessorios' 'pcs'
'casa_construcao' 'relogios_presentes'
'construcao_ferramentas_construcao' 'pet_shop' 'eletroportateis'
'agro_industria_e_comercio' 'moveis_sala' 'sinalizacao_e_seguranca'
'climatizacao' 'consoles_games' 'livros_interesse_geral'
'construcao_ferramentas_ferramentas' 'fashion_underwear_e_moda_praia'
'fashion_roupa_masculina'
'moveis_cozinha_area_de_servico_jantar_e_jardim'
'industria_comercio_e_negocios' 'telefonia_fixa'
'construcao_ferramentas_iluminacao' 'livros_tecnicos'
'eletrodomesticos_2' 'artigos_de_festas' 'bebidas' 'market_place'
'la_cuisine' 'construcao_ferramentas_jardim' 'fashion_roupa_feminina'
'casa_conforto' 'audio' 'alimentos_bebidas' 'musica' 'alimentos'
'tablets_impressao_imagem' 'livros_importados'
'portateis_casa_forno_e_cafe' 'fashion_esporte' 'artigos_de_natal'
'fashion_roupa_infanto_juvenil' 'dvds_blu_ray' 'artes_e_artesanato'
'pc_gamer' 'moveis_quarto' 'cine_foto' 'fraldas_higiene' 'flores'
'casa_conforto_2' 'portateis_cozinha_e_preparadores_de_alimentos'
'seguros_e_servicos' 'moveis_colchao_e_estofado' 'cds_dvds_musicais']

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 71 entries, 0 to 70
Data columns (total 2 columns):

#	Column	Non-Null Count	Dtype
0	product_category_name	71 non-null	object
1	product_category_name_english	71 non-null	object

dtypes: object(2)

memory usage: 1.2+ KB

None

['beleza_saude' 'informatica_acessorios' 'automotivo' 'cama_mesa_banho'
 'moveis_decoracao' 'esporte_lazer' 'perfumaria' 'utilidades_domesticas'
 'telefonia' 'relogios_presentes' 'alimentos_bebidas' 'bebes' 'papelaria'
 'tablets_impressao_imagem' 'brinquedos' 'telefonia_fixa'
 'ferramentas_jardim' 'fashion_bolsas_e_acessorios' 'eletroportateis'
 'consoles_games' 'audio' 'fashion_calcados' 'cool_stuff'
 'malas_acessorios' 'climatizacao' 'construcao_ferramentas_construcao'
 'moveis_cozinha_area_de_servico_jantar_e_jardim'
 'construcao_ferramentas_jardim' 'fashion_roupa_masculina' 'pet_shop'
 'moveis_escritorio' 'market_place' 'eletronicos' 'eletrodomesticos'
 'artigos_de_festas' 'casa_conforto' 'construcao_ferramentas_ferramentas'
 'agro_industria_e_comercio' 'moveis_colchao_e_estofado' 'livros_tecnicos'
 'casa_construcao' 'instrumentos_musicais' 'moveis_sala'
 'construcao_ferramentas_iluminacao' 'industria_comercio_e_negocios'
 'alimentos' 'artes' 'moveis_quarto' 'livros_interesse_geral'
 'construcao_ferramentas_seguranca' 'fashion_underwear_e_moda_praia'
 'fashion_esporte' 'sinalizacao_e_seguranca' 'pcs' 'artigos_de_natal'
 'fashion_roupa_feminina' 'eletrodomesticos_2' 'livros_importados'
 'bebidas' 'cine_foto' 'la_cuisine' 'musica' 'casa_conforto_2'
 'portateis_casa_forno_e_cafe' 'cds_dvds_musicais' 'dvds_blu_ray' 'flores'
 'artes_e_artesanato' 'fraldas_higiene' 'fashion_roupa_infanto_juvenil'
 'seguros_e_servicos']

Missing categories: {'pc_gamer', 'portateis_cozinha_e_preparadores_de_alimentos'}

['beleza_saude' 'informatica_acessorios' 'automotivo' 'cama_mesa_banho'

'moveis_decoracao' 'esporte_lazer' 'perfumaria' 'utilidades_domesticas'
'telefonia' 'relogios_presentes' 'alimentos_bebidas' 'bebes' 'papelaria'
'tablets_impressao_imagem' 'brinquedos' 'telefonia_fixa'
'ferramentas_jardim' 'fashion_bolsas_e_acessorios' 'eletroportateis'
'consoles_games' 'audio' 'fashion_calcados' 'cool_stuff'
'malas_acessorios' 'climatizacao' 'construcao_ferramentas_construcao'
'moveis_cozinha_area_de_servico_jantar_e_jardim'
'construcao_ferramentas_jardim' 'fashion_roupa_masculina' 'pet_shop'
'moveis_escritorio' 'market_place' 'eletronicos' 'eletrodomesticos'
'artigos_de_festas' 'casa_conforto' 'construcao_ferramentas_ferramentas'
'agro_industria_e_comercio' 'moveis_colchao_e_estofado' 'livros_tecnicos'
'casa_construcao' 'instrumentos_musicais' 'moveis_sala'
'construcao_ferramentas_iluminacao' 'industria_comercio_e_negocios'
'alimentos' 'artes' 'moveis_quarto' 'livros_interesse_geral'
'construcao_ferramentas_seguranca' 'fashion_underwear_e_moda_praia'
'fashion_esporte' 'sinalizacao_e_seguranca' 'pcs' 'artigos_de_natal'
'fashion_roupa_feminina' 'eletrodomesticos_2' 'livros_importados'
'bebidas' 'cine_foto' 'la_cuisine' 'musica' 'casa_conforto_2'
'portateis_casa_forno_e_cafe' 'cds_dvds_musicais' 'dvds_blu_ray' 'flores'
'artes_e_artesanato' 'fraldas_higiene' 'fashion_roupa_infanto_juvenil'
'seguros_e_servicos' 'portateis_cozinha_e_preparadores_de_alimentos'
'pc_gamer']

	product_id	product_category_name
0	1e9e8ef04dbcff4541ed26657ea517e5	perfumaria
1	3aa071139cb16b67ca9e5dea641aaa2f	artes
2	96bd76ec8810374ed1b65e291975717f	esporte_lazer
3	cef67bcfe19066a932b7673e239eb23d	bebes

4	9dc1a7de274444849c219cff195d0b71	utilidades_domesticas
5	41d3672d4792049fa1779bb35283ed13	instrumentos_musicais
6	732bd381ad09e530fe0a5f457d81becb	cool_stuff
7	2548af3e6e77a690cf3eb6368e9ab61e	moveis_decoracao
8	37cc742be07708b53a98702e77a21a02	eletrodomesticos
9	8c92109888e8cdf9d66dc7e463025574	brinquedos

	product_name_length	product_description_length	product_photos_qty	\
0	40.0	287.0	1.0	
1	44.0	276.0	1.0	
2	46.0	250.0	1.0	
3	27.0	261.0	1.0	
4	37.0	402.0	4.0	
5	60.0	745.0	1.0	
6	56.0	1272.0	4.0	
7	56.0	184.0	2.0	
8	57.0	163.0	1.0	
9	36.0	1156.0	1.0	

	product_weight_g	product_length_cm	product_height_cm	product_width_cm	\
0	225.0	16.0	10.0	14.0	
1	1000.0	30.0	18.0	20.0	
2	154.0	18.0	9.0	15.0	
3	371.0	26.0	4.0	26.0	
4	625.0	20.0	17.0	13.0	
5	200.0	38.0	5.0	11.0	
6	18350.0	70.0	24.0	44.0	

7	900.0	40.0	8.0	40.0
8	400.0	27.0	13.0	17.0
9	600.0	17.0	10.0	12.0

	product_category_name_english
0	perfumery
1	art
2	sports_leisure
3	baby
4	housewares
5	musical_instruments
6	cool_stuff
7	furniture_decor
8	home_appliances
9	toys

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 32336 entries, 0 to 32335

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	product_id	32336 non-null	object
1	product_category_name	32336 non-null	object
2	product_name_length	32336 non-null	float64
3	product_description_length	32336 non-null	float64
4	product_photos_qty	32336 non-null	float64
5	product_weight_g	32336 non-null	float64
6	product_length_cm	32336 non-null	float64

```
 7   product_height_cm            32336 non-null  float64
 8   product_width_cm             32336 non-null  float64
 9   product_category_name_english  32336 non-null  object
dtypes: float64(7), object(3)
memory usage: 2.5+ MB
None
                        seller_id  seller_zip_code_prefix  \
0  3442f8959a84dea7ee197c632cb2df15                   13023
1  d1b65fc7debc3361ea86b5f14c68d2e2                   13844
2  ce3ad9de960102d0677a81f5d0bb7b2d                   20031
3  c0f3eea2e14555b6faeea3dd58c1b1c3                    4195
4  51a04a8a6bdcb23deccc82b0b80742cf                   12914

          seller_city seller_state
0           campinas           SP
1         mogi guacu           SP
2      rio de janeiro           RJ
3           sao paulo           SP
4  braganca paulista           SP
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3095 entries, 0 to 3094
Data columns (total 4 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   seller_id               3095 non-null   object
 1   seller_zip_code_prefix  3095 non-null   int64
 2   seller_city             3095 non-null   object
```

```
 3   seller_state            3095 non-null   object
dtypes: int64(1), object(3)
memory usage: 96.8+ KB
None
       seller_zip_code_prefix
count             3095.000000
mean             32291.059451
std              32713.453830
min               1001.000000
25%               7093.500000
50%              14940.000000
75%              64552.500000
max              99730.000000
seller_id                 0
seller_zip_code_prefix    0
seller_city               0
seller_state              0
dtype: int64
Empty DataFrame
Columns: [seller_id, seller_zip_code_prefix, seller_city, seller_state]
Index: []
seller_city
sao paulo                 694
curitiba                  127
rio de janeiro             96
belo horizonte             68
ribeirao preto             52
```

...

ipua 1

muqui 1

timoteo 1

pouso alegre 1

rio de janeiro, rio de janeiro, brasil 1

Name: count, Length: 611, dtype: int64

2486

seller_city

sao paulo 695

curitiba 127

rio de janeiro 96

belo horizonte 68

ribeirao preto 52

...

ipua 1

muqui 1

timoteo 1

pouso alegre 1

rio de janeiro, rio de janeiro, brasil 1

Name: count, Length: 609, dtype: int64

['04482255', 'abadia de goias', 'afonso claudio', 'aguas claras df', 'alambari', 'almirante tamandare', 'alvares machado', 'alvorada', 'ampere', 'andira-pr', 'angra dos reis', 'angra dos reis rj', 'ao bernardo do campo', 'aparecida', 'aparecida de goiania', 'aperibe', 'aracaju', 'araquari', 'ararangua', 'arinos', "arraial d'ajuda (porto seguro)", 'arvorezinha', 'auriflama', 'auriflama/sp', 'avare', 'bage', 'bahia', 'balenario camboriu', 'bandeirantes', 'barbacena', 'barbacena/ minas gerais', 'barra velha', 'barrinha', 'barro alto', 'bebedouro', 'belo horizont', 'bertioga', 'bocaiuva do sul', 'bofete', 'bom jardim', 'bom jesus dos perdoes', 'bombinhas', 'bonfinopolis de minas', 'braco do norte', 'brasilia df', 'brejao', 'brotas', 'buritama', 'cacador', 'cachoeira do sul', 'caieiras', 'california', 'camanducaia', 'camboriu', 'campanha', 'campina das missoes', 'campina grande', 'campo do meio', 'campo

magro', 'campo mourao', 'campos dos goytacazes', 'campos novos', 'cananeia', 'carapicuiba / sao paulo', 'caratinga', 'cariacica / es', 'carmo da mata', 'carmo do cajuru', 'cascavael', 'castro pires', 'cataguases', 'caucaia', 'centro', 'cerqueira cesar', 'cianorte', 'clementina', 'colatina', 'colorado', 'concordia', 'condor', 'congonhal', 'congonhas', 'cordeiropolis', 'cordilheira alta', 'cornelio procopio', 'coxim', 'cravinhos', 'cruzeiro', 'descalvado', 'divisa nova', 'domingos martins', 'dracena', 'embu guacu', 'engenheiro coelho', 'entre rios do oeste', 'erechim', 'eunapolis', 'eusebio', 'extrema', 'feira de santana', 'fernando prestes', 'ferraz de vasconcelos', 'floranopolis', 'floresta', 'formosa do oeste', 'francisco morato', 'fronteira', 'gama', 'garulhos', 'garuva', 'goioere', 'governador valadares', 'guaimbe', 'guanambi', 'guanhaes', 'guara', 'guarapuava', 'guaratingueta', 'guariba', 'guiricema', 'holambra', 'horizontina', 'ibia', 'ibirite', 'icara', 'igrejinha', 'ilheus', 'ilicinea', 'imbe', 'imbituva', 'imigrante', 'ipaussu', 'ipe', 'ipua', 'irati', 'irece', 'itabira', 'itaborai', 'itapema', 'itapeva', 'itaporanga', 'itapui', 'itau de minas', 'itirapina', 'ivoti', 'jacarei / sao paulo', 'jaciara', 'jales', 'jambeiro', 'janauba', 'japira', 'jaragua', 'jarinu', 'ji parana', 'joao monlevade', 'joao pinheiro', 'jussara', 'juzeiro do norte', 'lages - sc', 'lagoa santa', 'laguna', 'lambari', 'laranjeiras do sul', 'leme', 'louveira', 'luiz alves', 'luziania', 'macatuba', 'mage', 'mairinque', 'mamanguape', 'manaus', 'mandaguacu', 'mandaguari', 'mandirituba', 'marapoama', 'marialva', 'marica', 'massaranduba', 'mateus leme', 'maua/sao paulo', 'medianeira', 'messias targino', 'miguelopolis', 'minas gerais', 'mineiros do tiete', 'mirandopolis', 'mococa', 'mogi das cruses', 'mogi das cruzes / sp', 'mombuca', 'monte alegre do sul', 'monte alto', 'monte siao', 'monteiro lobato', 'mucambo', 'muqui', 'muriae', 'neopolis', 'nhandeara', 'nova lima', 'nova petropolis', 'nova trento', 'novo hamburgo, rio grande do sul, brasil', 'novo horizonte', 'olimpia', 'oliveira', 'orlandia', 'orleans', 'ourinhos', 'ouro fino', 'ouro preto', 'pacatuba', 'paincandu', 'palotina', 'paracambi', 'parai', 'paraiba do sul', 'paraiso do sul', 'parana', 'paranavai', 'parnamirim', 'passos', 'pato bragado', 'pederneiras', 'pedregulho', 'pedrinhas paulista', 'pedro leopoldo', 'picarras', 'pilar do sul', 'pinhais/pr', 'pinhalao', 'piracanjuba', 'pirassununga', 'pirituba', 'pitangueiras', 'poa', 'ponte nova', 'portao', 'porto seguro', 'porto velho', 'portoferreira', 'pouso alegre', 'prados', 'presidente bernardes', 'presidente epitacio', 'presidente getulio', 'queimados', 'resende', 'ribeirao preto / sao paulo', 'ribeirao pretp', 'riberao preto', 'rio branco', 'rio das pedras', 'rio de janeiro / rio de janeiro', 'rio de janeiro \\rio de janeiro', 'rio de janeiro, rio de janeiro, brasil', 'rio do oeste', 'rio grande', 'rio negrinho', 'rio verde', 'robeirao preto', 'rolante', 'ronda alta', 's jose do rio preto', 'sabara', 'sando andre', 'santa catarina', 'santa cecilia', 'santa cruz do sul', 'santa maria da serra', 'santa rosa de viterbo', 'santa terezinha de goias', 'santo andre/sao paulo', 'santo angelo', 'santo antonio da patrulha', 'santo antonio de padua', 'santo antonio de posse', 'sao jose dos pinhais', 'sao paulo', 'sao bento', 'sao bernardo do capo', 'sao francisco do sul', 'sao joao da boa vista', 'sao joaquim da barra', 'sao jose do rio pret', 'sao jose dos pinhas', 'sao luis', "sao miguel d'oeste", 'sao miguel do oeste', 'sao paluo', 'sao paulo / sao paulo', 'sao paulo sp', 'sao paulop', 'sao pauo', 'sao pedro da aldeia', 'sao sebastiao', 'sao sebastiao da grama/sp', 'sao vicente', 'sapiranga', 'saquarema', 'sbc', 'sbc/sp', 'scao jose do rio pardo', 'serra redonda', 'serrana', 'sertanopolis', 'sinop', 'socorro', 'soledade', 'sp / sp', 'tabao da serra', 'taio', 'tambau', 'taruma', 'teixeira soares', 'teresina', 'terra boa', 'tiete', 'timoteo', 'tocantins', 'torres', 'tres coroas', 'tres de maio', 'tres rios', 'triunfo', 'ubatuba', 'uniao da vitoria', 'uruguaiana', 'vargem grande paulista',

'varzea alegre', 'varzea paulista', 'vassouras', 'vendas@creditparts.com.br', 'vera cruz', 'vespasiano', 'viana', 'vicente de carvalho', 'vitoria de santo antao', 'xaxim']

seller_state

SP 1849

PR 349

MG 244

SC 190

RJ 171

RS 129

GO 40

DF 30

ES 23

BA 19

CE 13

PE 9

PB 6

MS 5

RN 5

MT 4

RO 2

SE 2

AC 1

PI 1

MA 1

AM 1

PA 1

Name: count, dtype: int64

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 99441 entries, 0 to 99440

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	order_id	99441 non-null	object
1	customer_id	99441 non-null	object
2	order_status	99441 non-null	object
3	order_purchase_timestamp	99441 non-null	object
4	order_approved_at	99281 non-null	object
5	order_delivered_carrier_date	97658 non-null	object
6	order_delivered_customer_date	96476 non-null	object
7	order_estimated_delivery_date	99441 non-null	object

dtypes: object(8)

memory usage: 6.1+ MB

None

order_id 0

customer_id 0

order_status 0

order_purchase_timestamp 0

order_approved_at 160

order_delivered_carrier_date 1783

order_delivered_customer_date 2965

order_estimated_delivery_date 0

dtype: int64

Empty DataFrame

Columns: [order_id, customer_id, order_status, order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date]

Index: []

order_status

delivered 96478

shipped 1107

canceled 625

unavailable 609

invoiced 314

processing 301

created 5

approved 2

Name: count, dtype: int64

<class 'pandas.core.frame.DataFrame'>

Index: 1382 entries, 15 to 99406

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	order_id	1382 non-null	object
1	customer_id	1382 non-null	object
2	order_status	1382 non-null	object
3	order_purchase_timestamp	1382 non-null	datetime64[ns]
4	order_approved_at	1382 non-null	datetime64[ns]
5	order_delivered_carrier_date	1382 non-null	datetime64[ns]
6	order_delivered_customer_date	1373 non-null	datetime64[ns]
7	order_estimated_delivery_date	1382 non-null	datetime64[ns]

dtypes: datetime64[ns](5), object(3)

memory usage: 97.2+ KB

order_id 0

customer_id 0

order_status 0

order_purchase_timestamp 0

order_approved_at 0

order_delivered_carrier_date 0

order_delivered_customer_date 9

order_estimated_delivery_date 0

dtype: int64

<class 'pandas.core.frame.DataFrame'>

Index: 1373 entries, 15 to 99406

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	order_id	1373 non-null	object
1	customer_id	1373 non-null	object
2	order_status	1373 non-null	object
3	order_purchase_timestamp	1373 non-null	datetime64[ns]
4	order_approved_at	1373 non-null	datetime64[ns]
5	order_delivered_carrier_date	1373 non-null	datetime64[ns]
6	order_delivered_customer_date	1373 non-null	datetime64[ns]
7	order_estimated_delivery_date	1373 non-null	datetime64[ns]

dtypes: datetime64[ns](5), object(3)

memory usage: 96.5+ KB

order_id 0

customer_id 0

```
order_status                  0
order_purchase_timestamp      0
order_approved_at             0
order_delivered_carrier_date  0
order_delivered_customer_date 0
order_estimated_delivery_date 0
dtype: int64
15      dcb36b511fcac050b97cd5c05de84dc3
64      688052146432ef8253587b930b01a06d
199     58d4c4747ee059eeeb865b349b41f53a
210     412fccb2b44a99b36714bca3fef8ad7b
415     56a4ac10a4a8f2ba7693523bb439eede
                ...
99091   240ead1a7284667e0ec71d01f80e4d5e
99230   78008d03bd8ef7fcf1568728b316553c
99266   76a948cd55bf22799753720d4545dd2d
99377   a6bd1f93b7ff72cc348ca07f38ec4bee
99406   7fd85cb0143de098a4c5ab5a57bfbd91
Name: order_id, Length: 1373, dtype: object
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 98068 entries, 0 to 98067
Data columns (total 8 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   order_id                98068 non-null  object
 1   customer_id             98068 non-null  object
 2   order_status            98068 non-null  object
```

```
 3   order_purchase_timestamp       98068 non-null  datetime64[ns]
 4   order_approved_at              97908 non-null  datetime64[ns]
 5   order_delivered_carrier_date   96285 non-null  datetime64[ns]
 6   order_delivered_customer_date  95103 non-null  datetime64[ns]
 7   order_estimated_delivery_date  98068 non-null  datetime64[ns]
dtypes: datetime64[ns](5), object(3)
memory usage: 6.0+ MB
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 103886 entries, 0 to 103885
Data columns (total 5 columns):
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   order_id             103886 non-null  object
 1   payment_sequential    103886 non-null  int64
 2   payment_type          103886 non-null  object
 3   payment_installments  103886 non-null  int64
 4   payment_value         103886 non-null  float64
dtypes: float64(1), int64(2), object(2)
memory usage: 4.0+ MB
None
       payment_sequential  payment_installments  payment_value
count       103886.000000         103886.000000  103886.000000
mean             1.092679              2.853349     154.100380
std              0.706584              2.687051     217.494064
min              1.000000              0.000000       0.000000
25%              1.000000              1.000000      56.790000
```

50% 1.000000 1.000000 100.000000

75% 1.000000 4.000000 171.837500

max 29.000000 24.000000 13664.080000

order_id 0

payment_sequential 0

payment_type 0

payment_installments 0

payment_value 0

dtype: int64

Empty DataFrame

Columns: [order_id, payment_sequential, payment_type, payment_installments, payment_value]

Index: []

payment_type

credit_card 76795

bank_slip 19784

voucher 5775

debit_card 1529

not_defined 3

Name: count, dtype: int64

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 103886 entries, 0 to 103885

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	order_id	103886 non-null	object
1	payment_sequential	103886 non-null	int64
2	payment_type	103886 non-null	object

```
 3   payment_installments  103886 non-null  int64
 4   payment_value         103886 non-null  float64
dtypes: float64(1), int64(2), object(2)
memory usage: 4.0+ MB
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 98068 entries, 0 to 98067
Data columns (total 8 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   order_id                     98068 non-null  object
 1   customer_id                  98068 non-null  object
 2   order_status                 98068 non-null  object
 3   order_purchase_timestamp     98068 non-null  object
 4   order_approved_at            98068 non-null  object
 5   order_delivered_carrier_date  98068 non-null  object
 6   order_delivered_customer_date  98068 non-null  object
 7   order_estimated_delivery_date  98068 non-null  object
dtypes: object(8)
memory usage: 6.0+ MB
None
3
<class 'pandas.core.frame.DataFrame'>
Index: 103883 entries, 0 to 103885
Data columns (total 5 columns):
 #   Column              Non-Null Count   Dtype
---  ------              --------------   -----
```

```
 0   order_id               103883 non-null  object
 1   payment_sequential     103883 non-null  int64
 2   payment_type           103883 non-null  object
 3   payment_installments   103883 non-null  int64
 4   payment_value          103883 non-null  float64
dtypes: float64(1), int64(2), object(2)
memory usage: 4.8+ MB
None
<class 'pandas.core.frame.DataFrame'>
Index: 98065 entries, 0 to 98067
Data columns (total 8 columns):
 #   Column                         Non-Null Count  Dtype
---  ------                         --------------  -----
 0   order_id                       98065 non-null  object
 1   customer_id                    98065 non-null  object
 2   order_status                   98065 non-null  object
 3   order_purchase_timestamp       98065 non-null  object
 4   order_approved_at              98065 non-null  object
 5   order_delivered_carrier_date   98065 non-null  object
 6   order_delivered_customer_date  98065 non-null  object
 7   order_estimated_delivery_date  98065 non-null  object
dtypes: object(8)
memory usage: 6.7+ MB
None
payment_type
credit_card     76795
bank_slip       19784
```

```
voucher         5775
debit_card      1529
Name: count, dtype: int64
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99224 entries, 0 to 99223
Data columns (total 7 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   review_id               99224 non-null  object
 1   order_id                99224 non-null  object
 2   review_score            99224 non-null  int64
 3   review_comment_title    11568 non-null  object
 4   review_comment_message  40977 non-null  object
 5   review_creation_date    99224 non-null  object
 6   review_answer_timestamp  99224 non-null  object
dtypes: int64(1), object(6)
memory usage: 5.3+ MB
None
       review_score
count  99224.000000
mean       4.086421
std        1.347579
min        1.000000
25%        4.000000
50%        5.000000
75%        5.000000
max        5.000000
```

review_id 0

order_id 0

review_score 0

review_comment_title 87656

review_comment_message 58247

review_creation_date 0

review_answer_timestamp 0

dtype: int64

Empty DataFrame

Columns: [review_id, order_id, review_score, review_comment_title, review_comment_message, review_creation_date, review_answer_timestamp]

Index: []

Reviews before cleaning: 99224

Removed 814 duplicate reviews

Remaining: 98410

	review_id	order_id	\
0	7bc2406110b926393aa56f80a40eba40	73fc7af87114b39712e6da79b0a377eb	
1	80e641a11e56f04c1ad469d5645fdfde	a548910a1c6147796b98fdf73dbeba33	
2	228ce5500dc1d8e020d8d1322874b6f0	f9e4b658b201a9f2ecdecbb34bed034b	
3	e64fb393e7b32834bb789ff8bb30750e	658677c97b385a9be170737859d3511b	
4	f7c4243c7fe1938f181bec41a392bdeb	8e6bfb81e283fa7e4f11123a3fb894f1	

	review_score	review_comment_title	\
0	4	NaN	
1	5	NaN	
2	5	NaN	
3	5	NaN	

4 5 NaN

review_comment_message review_creation_date \

0 nan 2018-01-18

1 nan 2018-03-10

2 nan 2018-02-17

3 Recebi bem antes do prazo estipulado. 2017-04-21

4 Parabens lojas lannister adorei comprar pela I... 2018-03-01

review_answer_timestamp

0 2018-01-18 21:46:59

1 2018-03-11 03:05:13

2 2018-02-18 14:36:24

3 2017-04-21 22:02:06

4 2018-03-02 10:26:53

review_id order_id \

0 7bc2406110b926393aa56f80a40eba40 73fc7af87114b39712e6da79b0a377eb

1 80e641a11e56f04c1ad469d5645fdfde a548910a1c6147796b98fdf73dbeba33

2 228ce5500dc1d8e020d8d1322874b6f0 f9e4b658b201a9f2ecdecbb34bed034b

3 e64fb393e7b32834bb789ff8bb30750e 658677c97b385a9be170737859d3511b

4 f7c4243c7fe1938f181bec41a392bdeb 8e6bfb81e283fa7e4f11123a3fb894f1

review_score review_comment_title \

0 4 nan

1 5 nan

2 5 nan

3 5 nan

4 5 nan

review_comment_message review_creation_date \

0 nan 2018-01-18

1 nan 2018-03-10

2 nan 2018-02-17

3 Recebi bem antes do prazo estipulado. 2017-04-21

4 Parabens lojas lannister adorei comprar pela I... 2018-03-01

review_answer_timestamp

0 2018-01-18 21:46:59

1 2018-03-11 03:05:13

2 2018-02-18 14:36:24

3 2017-04-21 22:02:06

4 2018-03-02 10:26:53

/tmp/ipython-input-3249029422.py:277: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

dim_reviews_df[col] = dim_reviews_df[col].dt.strftime('%Y-%m-%d %H:%M:%S').fillna('')

/tmp/ipython-input-3249029422.py:277: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
  dim_reviews_df[col] = dim_reviews_df[col].dt.strftime('%Y-%m-%d %H:%M:%S').fillna('')
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000163 entries, 0 to 1000162
Data columns (total 5 columns):
 #   Column                    Non-Null Count    Dtype
---  ------                    --------------    -----
 0   geolocation_zip_code_prefix  1000163 non-null  int64
 1   geolocation_lat           1000163 non-null  float64
 2   geolocation_lng           1000163 non-null  float64
 3   geolocation_city          1000163 non-null  object
 4   geolocation_state         1000163 non-null  object
dtypes: float64(2), int64(1), object(2)
memory usage: 38.2+ MB
None
       geolocation_zip_code_prefix  geolocation_lat  geolocation_lng
count                 1.000163e+06     1.000163e+06     1.000163e+06
mean                  3.657417e+04    -2.117615e+01    -4.639054e+01
std                   3.054934e+04     5.715866e+00     4.269748e+00
min                   1.001000e+03    -3.660537e+01    -1.014668e+02
25%                   1.107500e+04    -2.360355e+01    -4.857317e+01
50%                   2.653000e+04    -2.291938e+01    -4.663788e+01
75%                   6.350400e+04    -1.997962e+01    -4.376771e+01
max                   9.999000e+04     4.506593e+01     1.211054e+02
geolocation_zip_code_prefix    0
geolocation_lat                0
geolocation_lng                0
geolocation_city               0
```

```
geolocation_state            0
dtype: int64
        geolocation_zip_code_prefix  geolocation_lat  geolocation_lng  \
15                             1046       -23.546081       -46.644820
44                             1046       -23.546081       -46.644820
65                             1046       -23.546081       -46.644820
66                             1009       -23.546935       -46.636588
67                             1046       -23.546081       -46.644820
...                             ...              ...              ...
1000153                       99970       -28.343273       -51.873734
1000154                       99950       -28.070493       -52.011342
1000159                       99900       -27.877125       -52.224882
1000160                       99950       -28.071855       -52.014716
1000162                       99950       -28.070104       -52.018658

        geolocation_city geolocation_state
15             sao paulo               SP
44             sao paulo               SP
65             sao paulo               SP
66             sao paulo               SP
67             sao paulo               SP
...                  ...              ...
1000153          ciriaco               RS
1000154         tapejara               RS
1000159   getulio vargas               RS
1000160         tapejara               RS
1000162         tapejara               RS
```

[261831 rows x 5 columns]

geolocation_zip_code_prefix

24220 1146

24230 1102

38400 965

35500 907

11680 879

...

72669 1

29014 1

72875 1

72873 1

72230 1

Name: count, Length: 19015, dtype: int64

geolocation_city

sao paulo 135800

rio de janeiro 62151

belo horizonte 27805

são paulo 24918

curitiba 16593

...

tres irmaos 1

morro chato 1

valao do barro 1

jordanésia 1

são valentim 1

Name: count, Length: 8011, dtype: int64

geolocation_state

SP 404268

MG 126336

RJ 121169

RS 61851

PR 57859

SC 38328

BA 36045

GO 20139

ES 16748

PE 16432

DF 12986

MT 12031

CE 11674

PA 10853

MS 10431

MA 7853

PB 5538

RN 5041

PI 4549

AL 4183

TO 3576

SE 3563

RO 3478

AM 2432

AC 1301

```
AP       853
RR       646
Name: count, dtype: int64
geolocation_city
sao paulo               160718
rio de janeiro           62151
belo horizonte           27805
curitiba                 16593
porto alegre             13521
                          ...
bicuiba                      1
sao miguel do aleixo         1
sa£o paulo                   1
canhoba                      1
muribeca                     1
Name: count, Length: 5968, dtype: int64
(112650, 7)
       order_item_id          price  freight_value
count  112650.000000  112650.000000  112650.000000
mean        1.197834     120.653739      19.990320
std         0.705124     183.633928      15.806405
min         1.000000       0.850000       0.000000
25%         1.000000      39.900000      13.080000
50%         1.000000      74.990000      16.260000
75%         1.000000     134.900000      21.150000
max        21.000000    6735.000000     409.680000
order_id              0
```

order_item_id 0

product_id 0

seller_id 0

shipping_limit_date 0

price 0

freight_value 0

dtype: int64

Empty DataFrame

Columns: [order_id, order_item_id, product_id, seller_id, shipping_limit_date, price, freight_value]

Index: []

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 112650 entries, 0 to 112649

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	order_id	112650 non-null	object
1	order_item_id	112650 non-null	int64
2	product_id	112650 non-null	object
3	seller_id	112650 non-null	object
4	shipping_deadline	112650 non-null	datetime64[ns]
5	price	112650 non-null	float64
6	freight_value	112650 non-null	float64

dtypes: datetime64[ns](1), float64(2), int64(1), object(3)

memory usage: 6.0+ MB

Removing Orders which are not linked to Order Items

order_id order_item_id \

```
7       000576fe39319847cbb9d288c5617fa6             1
56      002175704e8b209f61b9ad5cfd92b60e             1
67      002834535f7a609a5c68266f173fa59e             1
118     004345d16a1ab2c21962992c721c8643             1
283     00b4c651133c06fb175123a492f1cac3             1
...                          ...          ...
112471  ff9475663788de57817b221392697cce             1
112545  ffc0249fed109d5d056d7c79b7fa7dd9              1
112564  ffcc17a2f3331d4d25cf692060921690              1
112624  fff0db5573c78c1cb5a2b68a2bbd8d4a              1
112633  fff8286f77788ab8b55b2e5747fa7dd8              1

                        product_id                        seller_id  \
7       557d850972a7d6f792fd18ae1400d9b6  5996cddab893a4652a15592fb58ab8db
56      e6b6e13cf71449a457269f425b89dc74  b2ba3715d723d245138f291a6fe42594
67      df3655ac9aa8c6cbfa63bdd8d3b09c50  ea8482cd71df3c1969d7b9473ff13abc
118     d0fb1e667e989933a80444f93da833c0  0be8ff43f22e456b4e0371b2245e4d01
283     87ea2f45559e10519513a20b24147316  6a38087bc8ad4f89ff453561005f6dea
...                          ...                          ...
112471  f7ee078cfab4ad9144e1d75fd0258c5b  d57e18d5f73c7ccb7f7339b61166898d
112545  a05ed2eb565616b13f6d58a44cf7097a  7ff588a03c2aeae4fbd23f9ae64b760d
112564  42ec84ace63b58b8c5a7ba7be01d5fb8  2a84855fd20af891be03bc5924d2b453
112624  681953ccd5c33207d75571a4bfbe127d  777a0c55737f34ffeb78010f7542ab41
112633  a2da86fa759178e9e58e54aa1a144e59  ea8482cd71df3c1969d7b9473ff13abc

        shipping_deadline   price  freight_value
7       2018-07-10 12:30:45  810.00          70.75
```

56 2018-04-26 12:30:57 109.90 13.21

67 2018-07-26 17:44:36 37.99 19.18

118 2018-07-10 07:31:33 37.90 15.37

283 2018-08-22 14:27:21 158.90 14.59

... ...

112471 2018-04-26 02:31:46 37.90 16.32

112545 2018-08-15 14:45:15 24.99 7.44

112564 2018-07-03 14:50:16 89.90 13.46

112624 2018-07-03 09:31:23 69.95 7.75

112633 2018-07-05 22:31:13 24.99 15.28

[1592 rows x 7 columns]

(111058, 7)

Removing Products which are not linked to Order Items

order_id order_item_id \

123 0046e1d57f4c07c8c92ab26be8c3dfc0 1

125 00482f2670787292280e0a8153d82467 1

132 004f5d8f238e8908e6864b874eda3391 1

142 0057199db02d1a5ef41bacbf41f8f63b 1

171 006cb7cafc99b29548d4f412c7f9f493 1

... ...

112306 ff24fec69b7f3d30f9dc1ab3aee7c179 1

112333 ff3024474be86400847879103757d1fd 1

112350 ff3a45ee744a7c1f8096d2e72c1a23e4 1

112438 ff7b636282b98e0aa524264b295ed928 1

112501 ffa5e4c604dea4f0a59d19cc2322ac19 2

	product_id	seller_id
123	ff6caf9340512b8bf6d2a2a6df032cfa	38e6dada03429a47197d5d584d793b41
125	a9c404971d1a5b1cbc2e4070e02731fd	702835e4b785b67a084280efca355756
132	5a848e4ab52fd5445cdc07aab1c40e48	c826c40d7b19f62a09e2d7c5e7295ee2
142	41eee23c25f7a574dfaf8d5c151dbb12	e5a3438891c0bfdb9394643f95273d8e
171	e10758160da97891c2fdcbc35f0f031d	323ce52b5b81df2cd804b017b7f09aa7
...	...	...
112306	5a848e4ab52fd5445cdc07aab1c40e48	c826c40d7b19f62a09e2d7c5e7295ee2
112333	f9b1795281ce51b1cf39ef6d101ae8ab	3771c85bac139d2344864ede5d9341e3
112350	b61d1388a17e3f547d2bc218df02335b	07017df32dc5f2f1d2801e579548d620
112438	431df35e52c10451171d8037482eeb43	6cd68b3ed6d59aaa9fece558ad360c0a
112501	bd421826916d3e1d445cb860cea3c0fb	59cd88080b93f3c18508673122d26169

	shipping_deadline	price	freight_value
123	2017-10-02 15:49:17	7.79	7.78
125	2017-02-17 16:18:07	7.60	10.96
132	2018-03-06 09:29:25	122.99	15.61
142	2018-01-25 09:07:51	20.30	16.79
171	2018-02-22 13:35:28	56.00	14.14
...	...	...	...
112306	2018-02-01 02:40:12	122.99	15.61
112333	2017-11-21 03:55:39	39.90	9.94
112350	2017-05-10 10:15:19	139.00	21.42
112438	2018-02-22 15:35:35	49.90	15.11
112501	2017-12-11 08:41:20	29.99	15.10

[1602 rows x 7 columns]

(109456, 7)

Removing Seller which are not linked to Order Items

Empty DataFrame

Columns: [order_id, order_item_id, product_id, seller_id, shipping_deadline, price, freight_value]

Index: []

(109456, 7)

Output of Step 8

```
import snowflake.connector

import os

import logging

import pandas as pd


# Configure logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')


def run_post_validation_checks(snowflake_account, snowflake_user, snowflake_password, snowflake_warehouse, snowflake_database, snowflake_schema):

    """

    Connects to Snowflake and runs post-validation checks on the loaded data.


    Args:

      snowflake_account: Your Snowflake account identifier.

      snowflake_user: Your Snowflake username.

      snowflake_password: Your Snowflake password.

      snowflake_warehouse: The Snowflake warehouse to use.

      snowflake_database: The Snowflake database.
```

```
  snowflake_schema: The Snowflake schema.
"""
conn = None
cursor = None
try:
    logging.info("Attempting to connect to Snowflake for post-validation checks...")
    conn = snowflake.connector.connect(
        account=snowflake_account,
        user=snowflake_user,
        password=snowflake_password,
        warehouse=snowflake_warehouse,
        database=snowflake_database,
        schema=snowflake_schema
    )

    logging.info("Successfully connected to Snowflake for post-validation checks.")
    cursor = conn.cursor()

    print("\n--- Starting Post-Validation Checks ---")

    # 1. Verify Row Counts (Example - you'll need to compare with source counts)
    print("\n1. Verifying Row Counts...")
    table_row_counts_query = """
    SELECT table_name, row_count
    FROM information_schema.tables
    WHERE table_schema = UPPER('{snowflake_schema}')
    ORDER BY table_name;
```

```
""".format(snowflake_schema=snowflake_schema)


cursor.execute(table_row_counts_query)
row_counts = cursor.fetchall()
print("Row counts in Snowflake tables:")
for row in row_counts:
    print(f"  - {row[0]}: {row[1]} rows")
print("Note: Compare these counts with your source data counts.")


# 2. Data Quality Checks (Nulls, Duplicates, Data Ranges)
print("\n2. Running Data Quality Checks...")


# Check for null values in Primary Key columns
print("\n   - Checking for null values in Primary Key columns:")
pk_null_check_query = """
SELECT 'Dim_Customers', 'customer_id', COUNT(*) FROM Dim_Customers WHERE customer_id IS NULL
UNION ALL
SELECT 'Dim_Products', 'product_id', COUNT(*) FROM Dim_Products WHERE product_id IS NULL
UNION ALL
SELECT 'Dim_Sellers', 'seller_id', COUNT(*) FROM Dim_Sellers WHERE seller_id IS NULL
UNION ALL
SELECT 'Dim_Orders', 'order_id', COUNT(*) FROM Dim_Orders WHERE order_id IS NULL
UNION ALL
SELECT 'Dim_Payments', 'order_id', COUNT(*) FROM Dim_Payments WHERE order_id IS NULL -- Check part of composite PK
```

```
    UNION ALL
    SELECT 'Dim_Payments', 'payment_sequential', COUNT(*) FROM Dim_Payments WHERE payment_sequential IS NULL -- Check part of composite PK
    UNION ALL
    SELECT 'Dim_Reviews', 'review_id', COUNT(*) FROM Dim_Reviews WHERE review_id IS NULL
    UNION ALL
    SELECT 'Dim_Geolocation', 'geolocation_zip_code_prefix', COUNT(*) FROM Dim_Geolocation WHERE geolocation_zip_code_prefix IS NULL -- Check part of composite PK
    UNION ALL
    SELECT 'Dim_Geolocation', 'geolocation_lat', COUNT(*) FROM Dim_Geolocation WHERE geolocation_lat IS NULL -- Check part of composite PK
    UNION ALL
    SELECT 'Dim_Geolocation', 'geolocation_lng', COUNT(*) FROM Dim_Geolocation WHERE geolocation_lng IS NULL -- Check part of composite PK
    UNION ALL
    SELECT 'Fact_Order_Items', 'order_id', COUNT(*) FROM Fact_Order_Items WHERE order_id IS NULL -- Check part of composite PK
    UNION ALL
    SELECT 'Fact_Order_Items', 'order_item_id', COUNT(*) FROM Fact_Order_Items WHERE order_item_id IS NULL; -- Check part of composite PK
    """
    cursor.execute(pk_null_check_query)
    pk_null_results = cursor.fetchall()
    for row in pk_null_results:
        if row[2] > 0:
            print(f"    - WARNING: Table '{row[0]}', Column '{row[1]}' has {row[2]} null values.")
        else:
```

```
            print(f"     - Table '{row[0]}', Column '{row[1]}' has no null values.")


    # Check for duplicate records based on Primary Keys
    print("\n   - Checking for duplicate records based on Primary Keys:")
    duplicate_check_queries = [
        ("""SELECT 'Dim_Customers' AS table_name, customer_id AS pk_value, COUNT(*) AS duplicate_count FROM Dim_Customers GROUP BY customer_id HAVING COUNT(*) > 1;""", ['Dim_Customers', 'customer_id']),
        ("""SELECT 'Dim_Products' AS table_name, product_id AS pk_value, COUNT(*) AS duplicate_count FROM Dim_Products GROUP BY product_id HAVING COUNT(*) > 1;""", ['Dim_Products', 'product_id']),
        ("""SELECT 'Dim_Sellers' AS table_name, seller_id AS pk_value, COUNT(*) AS duplicate_count FROM Dim_Sellers GROUP BY seller_id HAVING COUNT(*) > 1;""", ['Dim_Sellers', 'seller_id']),
        ("""SELECT 'Dim_Orders' AS table_name, order_id AS pk_value, COUNT(*) AS duplicate_count FROM Dim_Orders GROUP BY order_id HAVING COUNT(*) > 1;""", ['Dim_Orders', 'order_id']),
        ("""SELECT 'Dim_Payments' AS table_name, order_id, payment_sequential, COUNT(*) AS duplicate_count FROM Dim_Payments GROUP BY order_id, payment_sequential HAVING COUNT(*) > 1;""", ['Dim_Payments', 'order_id', 'payment_sequential']), # Composite PK
        ("""SELECT 'Dim_Reviews' AS table_name, review_id AS pk_value, COUNT(*) AS duplicate_count FROM Dim_Reviews GROUP BY review_id HAVING COUNT(*) > 1;""", ['Dim_Reviews', 'review_id']),
        ("""SELECT 'Dim_Geolocation' AS table_name, geolocation_zip_code_prefix, geolocation_lat, geolocation_lng, COUNT(*) AS duplicate_count FROM Dim_Geolocation GROUP BY geolocation_zip_code_prefix, geolocation_lat, geolocation_lng HAVING COUNT(*) > 1;""", ['Dim_Geolocation', 'geolocation_zip_code_prefix', 'geolocation_lat', 'geolocation_lng']), # Composite PK
        ("""SELECT 'Fact_Order_Items' AS table_name, order_id, order_item_id, COUNT(*) AS duplicate_count FROM Fact_Order_Items GROUP BY order_id, order_item_id HAVING COUNT(*) > 1;""", ['Fact_Order_Items', 'order_id', 'order_item_id']) # Composite PK
    ]
```

```
    for query, pk_cols in duplicate_check_queries:
        cursor.execute(query)
        duplicate_results = cursor.fetchall()
        if duplicate_results:
            table_name = duplicate_results[0][0] # Get table name from the first result row
            print(f"    - WARNING: Duplicates found in table '{table_name}' based on PK columns {pk_cols}. First few duplicates:")
            for row in duplicate_results[:5]: # Print up to 5 duplicate examples
                print(f"      - {row}")
        else:
            print(f"    - No duplicates found for table '{pk_cols[0]}'.")

    # Check for non-positive values in price and freight_value in Fact_Order_Items
    print("\n  - Checking for non-positive price/freight values in Fact_Order_Items:")
    price_freight_check_query = """
    SELECT 'Fact_Order_Items' AS table_name, 'price' AS column_name, COUNT(*) AS invalid_count FROM Fact_Order_Items WHERE price < 0
    UNION ALL
    SELECT 'Fact_Order_Items', 'freight_value', COUNT(*) FROM Fact_Order_Items WHERE freight_value < 0;
    """
    cursor.execute(price_freight_check_query)
    price_freight_results = cursor.fetchall()
    for row in price_freight_results:
        if row[2] > 0:
            print(f"    - WARNING: Table '{row[0]}', Column '{row[1]}' has {row[2]} non-positive values.")
        else:
```

```
            print(f"     - Table '{row[0]}', Column '{row[1]}' has no non-positive values.")


    # Check for invalid review scores
    print("\n   - Checking for invalid review scores in Dim_Reviews:")
    invalid_review_score_query = """
    SELECT 'Dim_Reviews' AS table_name, 'review_score' AS column_name, COUNT(*) AS invalid_count
    FROM Dim_Reviews
    WHERE review_score < 1 OR review_score > 5;
    """
    cursor.execute(invalid_review_score_query)
    invalid_review_results = cursor.fetchall()
    for row in invalid_review_results:
        if row[2] > 0:
            print(f"     - WARNING: Table '{row[0]}', Column '{row[1]}' has {row[2]} invalid scores (expected 1-5).")
        else:
            print(f"     - Table '{row[0]}', Column '{row[1]}' has no invalid scores.")



    # 3. Referential Integrity Checks (Orphaned Records)
    print("\n3. Running Referential Integrity Checks (checking for orphaned records)...")

    # Check for order_ids in Fact_Order_Items that do not exist in Dim_Orders
    print("   - Checking for orphaned records in Fact_Order_Items (order_id)...")
    orphan_order_check_query = """
    SELECT fo.order_id
    FROM Fact_Order_Items fo
```

```
    LEFT JOIN Dim_Orders d ON fo.order_id = d.order_id
    WHERE d.order_id IS NULL
    LIMIT 10; -- Limit results as there could be many
    """
    cursor.execute(orphan_order_check_query)
    orphan_order_results = cursor.fetchall()
    if orphan_order_results:
        print(f"     - WARNING: Found {len(orphan_order_results)} orphaned order_ids in Fact_Order_Items (first 10):")
        for row in orphan_order_results:
            print(f"       - {row[0]}")
    else:
        print("     - No orphaned order_ids found in Fact_Order_Items.")

    # Check for product_ids in Fact_Order_Items that do not exist in Dim_Products
    print("   - Checking for orphaned records in Fact_Order_Items (product_id)...")
    orphan_product_check_query = """
    SELECT fo.product_id
    FROM Fact_Order_Items fo
    LEFT JOIN Dim_Products dp ON fo.product_id = dp.product_id
    WHERE dp.product_id IS NULL
    LIMIT 10; -- Limit results as there could be many
    """
    cursor.execute(orphan_product_check_query)
    orphan_product_results = cursor.fetchall()
    if orphan_product_results:
        print(f"     - WARNING: Found {len(orphan_product_results)} orphaned product_ids in Fact_Order_Items (first 10):")
```

```
        for row in orphan_product_results:
            print(f"       - {row[0]}")
    else:
        print("     - No orphaned product_ids found in Fact_Order_Items.")

    # Check for seller_ids in Fact_Order_Items that do not exist in Dim_Sellers
    print("   - Checking for orphaned records in Fact_Order_Items (seller_id)...")
    orphan_seller_check_query = """
    SELECT fo.seller_id
    FROM Fact_Order_Items fo
    LEFT JOIN Dim_Sellers ds ON fo.seller_id = ds.seller_id
    WHERE ds.seller_id IS NULL
    LIMIT 10; -- Limit results as there could be many
    """
    cursor.execute(orphan_seller_check_query)
    orphan_seller_results = cursor.fetchall()
    if orphan_seller_results:
        print(f"     - WARNING: Found {len(orphan_seller_results)} orphaned seller_ids in
Fact_Order_Items (first 10):")
        for row in orphan_seller_results:
            print(f"       - {row[0]}")
    else:
        print("     - No orphaned seller_ids found in Fact_Order_Items.")


    print("\n--- Post-Validation Checks Finished ---")
```

```
    except Exception as e:
        logging.error(f"An error occurred during post-validation checks: {e}", exc_info=True)
        print(f'An error occurred during post-validation checks: {e}")


    finally:
        # Close the cursor and connection
        if cursor:
            cursor.close()
            logging.info("Snowflake cursor closed.")
        if conn:
            conn.close()
            logging.info("Snowflake connection closed after post-validation checks.")

# Define Snowflake connection details using placeholder environment variables
# In a real-world scenario, these would be set securely in your environment
# or using a secrets management tool.
SNOWFLAKE_ACCOUNT = os.getenv("SNOWFLAKE_ACCOUNT", "WZATSRO-TP51811")
SNOWFLAKE_USER = os.getenv("SNOWFLAKE_USER", "snehaumrikar")
SNOWFLAKE_PASSWORD = os.getenv("SNOWFLAKE_PASSWORD", "Snowflake12345")
SNOWFLAKE_WAREHOUSE = os.getenv("SNOWFLAKE_WAREHOUSE", "MBI_DW")
SNOWFLAKE_DATABASE = os.getenv("SNOWFLAKE_DATABASE", "OLIST_DB")
SNOWFLAKE_SCHEMA = os.getenv("SNOWFLAKE_SCHEMA", "OLIST_SCHEMA")

# Run the post-validation checks
```

```
if __name__ == "__main__":
    run_post_validation_checks(SNOWFLAKE_ACCOUNT, SNOWFLAKE_USER, SNOWFLAKE_PASSWORD, SNOWFLAKE_WAREHOUSE, SNOWFLAKE_DATABASE, SNOWFLAKE_SCHEMA)
```